TurfMate

A Friendly Guide to How the App Works

Written for curious minds who want to understand Flutter — no scary jargon, just simple examples!

■ Football Field Booking	■ Razorpay Payments	■ User Profiles	■ Flutter App

What's inside?

➤ Chapter 1 — The Big Picture: What is TurfMate?

➤ Chapter 2 — main.dart: The App's Front Door

➤ Chapter 3 — TurfField & TimeSlot: The Building Blocks

➤ Chapter 4 — Profile Screen: Showing Off Your Info

➤ Chapter 5 — Payment Screen: The Money Magic

➤ Chapter 6 — Key Concepts Cheatsheet

➤ Chapter 7 — Viva Voice: What to Say When Teacher Asks

➤ Chapter 8 — Alteration Questions: Quick Change Guide

■ Chapter 1

The Big Picture — What is TurfMate?

What is this app?

TurfMate is a **sports turf booking app** — think of it like Zomato, but instead of ordering food, you're booking a football or cricket ground for an hour. You browse available turfs, pick a time slot, pay online using Razorpay, and get a confirmation. Simple!

> **■ Real-World Analogy**
> Imagine you want to play cricket on Saturday evening. You open TurfMate, see 'Sunrise Arena' has a slot from 6 PM to 7 PM, tap Book, pay ■600 via UPI, and you're done! The app is the 'middleman' between you and the turf owner.

What is Flutter?

Flutter is Google's toolkit for building apps. The cool thing is — you write the code **once** and it runs on Android, iOS, and even the web! TurfMate is built with Flutter using the Dart programming language.

> **■ Flutter = LEGO**
> Think of Flutter like LEGO bricks. Each 'Widget' is a brick — some bricks are buttons, some are text, some are images. You stack them together to build a full screen. The entire TurfMate UI is made of these widgets stacked cleverly!

The App has 7 Screens

Screen	What it does
SplashScreen	The loading screen when app opens — like a movie intro
LoginScreen	Where you enter your email & password
HomeScreen	Main dashboard — shows available turfs
BookingScreen	Where you pick date, time slot, duration
ProfileScreen	Your name, email, booking stats, logout button
PaymentScreen	Checkout page — shows total, opens Razorpay
OwnerDashboard	Special screen only for turf owners to manage their fields

■ Chapter 2

main.dart — The App's Front Door

Why does main.dart matter?

Every Flutter app starts running from **main.dart**. It's like the reception desk of a hotel — the first thing you see when you walk in, and it directs you to the right room (screen). It sets up 3 important things: **State Management**, **Themes**, and **Routes**.

1. State Management with Provider

The app needs to remember things — like 'is the user logged in?' or 'which bookings has this user made?'. Provider is like a **shared whiteboard** that every screen can read and write to.

■ Provider = School Notice Board

Imagine a school notice board. Teachers (Providers) write important info on it. Any student (Widget) who walks past can read what's on the board. When the notice changes, everyone who was watching it gets notified automatically! That's exactly how Provider works in Flutter.

```
MultiProvider( // One giant notice board
providers: [
ChangeNotifierProvider(create: (_) => AuthProvider()), // User login info
ChangeNotifierProvider(create: (_) => BookingProvider()), // Booking data
ChangeNotifierProvider(create: (_) => CommunityProvider()),// Community stuff
],
child: TurfApp(), // The entire app sits 'below' the notice board
)
```

2. Routes — The App's Map

Routes are like a map of your app. Each route is a string (like a street name) linked to a screen. When you call **Navigator.pushNamed(context, '/payment')**, Flutter looks up '/payment' in the map and takes you to PaymentScreen.

Route (Street Name)	Screen (Destination)
'/'	SplashScreen — very first screen when app opens
'/login'	LoginScreen — enter your credentials
'/home'	HomeScreen — browse turfs
'/booking'	BookingScreen — pick slot and duration
'/profile'	ProfileScreen — your info

Route (Street Name)	Screen (Destination)
'/payment'	PaymentScreen — checkout and pay
'/owner'	OwnerDashboardScreen — for turf owners only

3. Theme — The App's Outfit

AppTheme is like choosing a uniform for your app. You define colours, button styles, and card shapes once — and **every single screen** automatically follows those rules. Change the primary colour in one place and the whole app changes!

```
static const Color primaryGreen = Color(0xFF2E7D32); // Dark forest green
static const Color accentOrange = Color(0xFFFF6F00); // Warm orange for highlights

// 0xFF = fully opaque (like 100% opacity)
// 2E7D32 = the hex colour code (a dark green)
```

Why two themes?

lightTheme = normal white background (day mode)
darkTheme = dark background (night mode)
themeMode: ThemeMode.system = Flutter auto-picks based on your phone's setting!

■■ Chapter 3

TurfField & TimeSlot — The Building Blocks

What is a Model?

A model is just a Dart class that holds data — like a form you fill in. Instead of keeping separate variables for a turf's name, price, and rating scattered everywhere, you bundle them into one neat class called **TurfField**.

■ Model = ID Card

Think of TurfField like an ID card for a sports field. Every field gets its own card with its name, location, price per hour, rating, amenities, and owner details. Instead of carrying 10 separate pieces of paper, you carry one card!

TurfField — Every Turf's ID Card

Field	What it stores
id	Unique code like 'field_001' — like an Aadhaar number for the field
name	Display name — e.g. 'Sunrise Arena'
type	Sport — 'Football', 'Cricket', 'Badminton'
pricePerHour	Cost in rupees — e.g. 600.0 for ■600/hr
rating	Average user rating — 0.0 to 5.0
amenities	List of facilities — ['Floodlights', 'Parking', 'Changing Room']
isAvailable	Can it be booked right now? true or false

final vs non-final — The 'Can I Change This?' Rule

Most fields are marked **final** — once set, they can't be changed. But **isAvailable** is NOT final, because a turf owner can take their field offline for maintenance. It makes sense for that to change!

final fields (locked after creation):

id — the ID never changes

name — field name stays the same

pricePerHour — set by owner once

rating — calculated, not editable directly

non-final fields (can change):

isAvailable — owner can turn off/on

isBooked (in TimeSlot) — changes when someone books

toJson() — Packing a Suitcase

When you want to send a TurfField's data to a server (like saving it to a database), you can't just send the Dart object — servers understand JSON (text format). toJson() converts the object into a Map, like packing your clothes into a suitcase.

```
Map<String, dynamic> toJson() {
return {
'id': id, // 'field_001'
'name': name, // 'Sunrise Arena'
'pricePerHour': pricePerHour, // 600.0
'amenities': amenities, // ['Floodlights', 'Parking']
// ...and so on
};
}
```

TimeSlot — One Hour on the Calendar

A TimeSlot is even simpler. It represents one bookable hour on the calendar. It stores the hour as a number (e.g., 17 means 5 PM in 24-hour format) and a human-readable label like '5:00 PM – 6:00 PM' that the user actually sees.

```
class TimeSlot {
final String id; // 'slot_17'
final int hour; // 17 (means 5 PM in 24-hour format)
final String label; // '5:00 PM - 6:00 PM' (what user sees)
bool isBooked; // false = available, true = taken
}
```

■ TimeSlot = Train Seat

Imagine a train with 24 seats — one for each hour of the day. Each seat (TimeSlot) has a seat number (hour = 17), a readable label ('5 PM window seat'), and a status — either free or occupied (isBooked). When someone books it, isBooked flips to true!

■ Chapter 4

Profile Screen — Showing Off Your Info

What does this screen show?

The Profile Screen shows the logged-in user's details — their name, email, number of bookings made, and settings like notifications and logout. It's purely for **displaying** information, not editing.

Why StatelessWidget?

ProfileScreen is a **StatelessWidget**. This means it has NO memory of its own — it doesn't store anything internally. All the data (name, email, bookings) comes from Providers. It's like a TV screen — it just displays what's given to it.

StatelessWidget (Profile Screen)	StatefulWidget (Payment Screen)
No internal memory	Has its own memory (state)
Just displays data from Providers	Tracks: idle / processing / success
Like a TV screen — shows input, can't store	Like a smartphone — can remember things

context.watch vs context.read

These two are how screens talk to Providers. Think of them like two different ways to check a notice board:

■ watch vs read

context.watch = Standing in front of the notice board and staring at it. The moment something changes, you see it instantly and your screen refreshes. context.read = Walking past the board, glancing once, and walking away. You get the current value but won't be notified of future changes. Use this inside button taps.

```
// In the build() method — use watch (auto-refreshes)
final auth = context.watch<AuthProvider>();
final bookings = context.watch<BookingProvider>().bookingsForCustomer(auth.userId);

// In a button tap handler — use read (one-time)
onTap: () {
context.read<AuthProvider>().logout(); // just do the action, no need to listen
Navigator.pushReplacementNamed(context, '/login');
}
```

The Green Gradient Banner

At the top of the profile screen is a dark-to-light green banner showing the user's avatar, name, and email. The avatar shows the **first letter** of the user's name inside a circle — like WhatsApp does when

you don't have a profile photo.

> **How the avatar letter is made**
> auth.userName[0] — takes the 0th character (first letter) of the name
> .toUpperCase() — makes it capital
> Example: if name is 'riya', it shows 'R' inside the circle

pushReplacementNamed — The One-Way Door

When you press Logout, the app uses **pushReplacementNamed(context, '/login')**. This is a one-way door — it takes you to login AND removes the profile screen from history. So pressing the back button after logout won't accidentally take you back to profile!

Navigation Method	Behaviour
pushNamed	Opens new screen. Back button returns to previous. Like opening a new door but keeping the old one.
pushReplacementNamed	Replaces current screen. No going back to it. Like logout — you don't want back to go back to profile.
pushNamedAndRemoveUntil	Opens new screen AND clears all history. Used after payment success — you don't want back to go back to payment.

■ Chapter 5

Payment Screen — The Money Magic

The most complex file in the project!

PaymentScreen is a **StatefulWidget** — it needs to remember what phase the payment is in. It handles the entire checkout experience: showing the booking summary, opening Razorpay, waiting for payment confirmation, and displaying the success receipt.

The 4 States of Payment

Think of the payment flow like a traffic signal with 4 stages:

State	What's Happening
idle	Waiting. The Pay button is visible and clickable. Nothing is happening yet.
processing	Button clicked! Contacting Razorpay's server to create an order. Button shows 'Opening Gateway...'
verifying	Razorpay payment done! Now confirming with our server to officially create the booking.
success	Done! Booking confirmed. Shows the green success receipt screen.

■ States = ATM Machine Steps

When you use an ATM: idle = card inserted, waiting. processing = 'Please wait, contacting bank...'. verifying = 'Authorizing transaction...'. success = 'Cash dispensed! Thank you!'. Each step is a different state and the screen shows different things at each step.

initState() and dispose() — Birth and Death

Every StatefulWidget has a lifecycle — it's born (initState) and it dies (dispose). We set up Razorpay listeners when the screen is born, and clean them up when it dies.

```
@override
void initState() { // Called ONCE when screen first appears
super.initState();
_razorpay = Razorpay();
_razorpay.on(Razorpay.EVENT_PAYMENT_SUCCESS, _onSuccess); // Register callbacks
_razorpay.on(Razorpay.EVENT_PAYMENT_ERROR, _onError);
}
@override
void dispose() { // Called ONCE when screen is removed
_razorpay.clear(); // MUST clean up! Otherwise memory leak!
super.dispose();
}
```

■ **dispose() = Cleaning Up After a Party**

Imagine you put up decorations (Razorpay listeners) for a party. When the party (screen) is over, you MUST take down the decorations. If you don't, they stay up forever taking space — that's a memory leak! dispose() is your cleanup crew.

The late Keyword

Notice **late Razorpay _razorpay**. The keyword 'late' is a promise to Dart: 'I'll definitely set this up before using it, just trust me!' We can't initialize it at the top because it needs initState() to run first.

```
// Without 'late' — ERROR! initState hasn't run yet at this point
Razorpay _razorpay = Razorpay(); // This runs before initState!
// With 'late' — CORRECT! We promise to set it up before using it
late Razorpay _razorpay; // Will be set in initState()
```

The mounted Check — Don't Talk to Ghosts!

After an async operation (like waiting for payment), the user might have pressed back and left the screen. If we call setState() on a screen that no longer exists — it crashes! The **mounted** check prevents this.

```
// WRONG — might crash if user left the screen
setState(() { _state = _PayState.success; });
// CORRECT — check first!
if (mounted) {
setState(() { _state = _PayState.success; });
}
```

■ **mounted = Checking if Someone's Still Home**

Imagine calling a friend on the phone, but they've already left the house. If you shout instructions to an empty house, nothing happens (or it crashes!). mounted checks if the screen is still 'home' before giving it instructions.

■ Chapter 6

Key Concepts Cheatsheet

Flutter Concepts in Plain English

Concept	Simple Explanation
StatelessWidget	A screen/component with NO internal memory. Gets all data from outside (Providers). Like a TV — shows what you give it.
StatefulWidget	A screen/component WITH its own memory. Can call setState() to update itself. Like a smartphone — remembers things.
Provider	A shared data store. Like a school notice board — anyone can read it, and everyone watching it gets notified of changes.
ChangeNotifier	The class you extend to make a Provider. Call notifyListeners() to shout 'HEY! I changed!' to all watching widgets.
context.watch()	Subscribe to a Provider AND auto-rebuild when it changes. Use in build() methods.
context.read()	Read a Provider once without subscribing. Use in button taps and event handlers.
initState()	Lifecycle method — runs once when widget is born. Set up resources here.
dispose()	Lifecycle method — runs once when widget is destroyed. Clean up resources here!
late	Dart keyword meaning 'I'll initialize this before using it.' For things you can't set up at declaration time.
String?	Nullable String — can hold a value OR null. The ? means 'this might be empty'.
toJson()	Converts a Dart object to a Map so it can be sent to a server as JSON text.
final	Field can only be set once (in constructor). After that it's locked forever.
mounted	Boolean — is this widget still on screen? Check before calling setState() after async operations.

Navigation Quick Reference

Code	What happens
Navigator.pushNamed(ctx, '/home')	Go to /home, can press back to return
Navigator.pushReplacementNamed(ctx, '/login')	Go to /login, CAN'T press back to previous screen
Navigator.pushNamedAndRemoveUntil(ctx, '/home', ...)	Go to /home, clears ALL history behind it
Navigator.pop(ctx)	Go back to the previous screen

■ Chapter 7

Viva Voice — What to Say When Teacher Asks

Answer these confidently!

If teacher asks...	You say...
"What is Provider?"	Provider is a state management solution. It uses Flutter's InheritedWidget under the hood. A ChangeNotifier class holds data and calls notifyListeners() when it changes — all widgets using context.watch() automatically rebuild.
"Why StatefulWidget for PaymentScreen?"	Because the payment flow has multiple UI states — idle, processing, verifying, success — that change based on async events from Razorpay. We need setState() to update the screen as each phase completes.
"Why StatelessWidget for ProfileScreen?"	Profile doesn't have local mutable state. All data comes from AuthProvider and BookingProvider. context.watch() handles auto-rebuilding whenever those providers change.
"What does toJson() do?"	It converts the Dart object into a Map (key-value dictionary) so it can be serialized to JSON format for API calls or database writes.
"What is initState() for?"	It's a lifecycle method called once when the StatefulWidget is first inserted into the widget tree. We use it to set up Razorpay event listeners before the UI builds.
"What does the mounted check do?"	It checks if the widget is still in the widget tree. After an async operation, the user might have navigated away — calling setState() on an unmounted widget throws an error, so we check mounted first.
"What is the late keyword?"	late tells Dart the variable will be initialized before it's first used, but not at declaration time. We use it for _razorpay because it's set up in initState(), not at field declaration.
"What happens if you remove _razorpay.clear() from dispose()?"	Memory leak! The Razorpay listeners hold references to the widget's state. Without clearing them, the garbage collector can't free the memory. It could also crash if a callback fires on a disposed widget.

If teacher asks...	You say...
"What is the difference between context.watch and context.read?"	context.watch subscribes to the provider and triggers a rebuild whenever it changes — use it in build(). context.read is a one-time access with no subscription — use it in event handlers and callbacks.
"What is Material Design 3?"	Material Design 3 (enabled via useMaterial3: true) is Google's latest UI design system. It features rounded corners, updated color schemes, and new component styles. Flutter supports it natively.

Chapter 8

Alteration Questions — Quick Change Guide

The teacher might ask you to change something live! Here's what to do:

Q1: Add an imageUrl field to TurfField

Where to change: **In turf_field.dart**

```
final String imageUrl; // Add as a field
required this.imageUrl, // Add to constructor
'imageUrl': imageUrl, // Add to toJson()
// Then use: Image.network(field.imageUrl) in UI
```

Effect

Every place creating a TurfField must now pass an imageUrl. The booking/home screen can show the field's photo.

Q2: Change theme color from green to blue

Where to change: **In main.dart AppTheme**

```
static const Color primaryGreen = Color(0xFF1565C0); // Dark blue
static const Color lightGreen = Color(0xFF42A5F5); // Light blue
// That's it! Whole app changes because everything inherits theme
```

Effect

The entire app instantly turns blue — AppBars, buttons, input borders — all change because they reference the theme.

Q3: Add fromJson() to TurfField

Where to change: **In turf_field.dart**

```
factory TurfField.fromJson(Map<String, dynamic> json) {
return TurfField(
id: json['id'],
name: json['name'],
pricePerHour: (json['pricePerHour'] as num).toDouble(),
amenities: List<String>.from(json['amenities']),
isAvailable: json['isAvailable'] ?? true,
// ... all other fields
);
}
```

Effect

Now you can parse JSON from an API response into a TurfField object. This is the counterpart of toJson().

Q4: Always use dark mode

Where to change: **In main.dart MaterialApp**

```
themeMode: ThemeMode.dark, // was ThemeMode.system
```

Effect

App always uses dark mode regardless of phone settings. darkTheme will be applied everywhere.

Q5: Add a new /settings route

Where to change: **Three steps needed**

```
// Step 1: Create screens/settings_screen.dart with SettingsScreen class
// Step 2: Import it in main.dart
// Step 3: Add to routes map:
'/settings': (_) => const SettingsScreen(),
// Step 4: Navigate with:
Navigator.pushNamed(context, '/settings');
```

Effect

A new page is reachable from anywhere in the app via the named route.

Q6: Show phone number on profile screen

Where to change: **In profile_screen.dart**

```
// After Text(auth.userEmail), add:
Text(
auth.userPhone ?? 'No phone set',
style: TextStyle(color: Colors.white70, fontSize: 13),
),
// If AuthProvider doesn't have userPhone, add it there too
```

Effect

Users now see their phone number on the profile page below their email.